

Algoritma Analizi

Büyüme hızlarının karşılaştırılması
(Özet)

$\approx, \preceq, \prec, \succeq$, and $\succ$

Properties of the relations:

$$f(n) \approx g(n) \Leftrightarrow f(n) = \Theta(g(n))$$

$$f(n) \preceq g(n) \Leftrightarrow f(n) = O(g(n))$$

$$f(n) \prec g(n) \Leftrightarrow f(n) = o(g(n))$$

$$f(n) \succeq g(n) \Leftrightarrow f(n) = \Omega(g(n))$$

$$f(n) \succ g(n) \Leftrightarrow f(n) = \omega(g(n))$$

1. Reflexivity: $f(n) \approx f(n)$, $f(n) \preceq f(n)$, $f(n) \succeq f(n)$.

2. Symmetry: $f(n) \approx g(n) \Leftrightarrow g(n) \approx f(n)$.

3. Transitivity:

$$f(n) \approx g(n) \text{ and } g(n) \approx h(n) \Rightarrow f(n) \approx h(n)$$

$$f(n) \preceq g(n) \text{ and } g(n) \preceq h(n) \Rightarrow f(n) \preceq h(n)$$

$$f(n) \prec g(n) \text{ and } g(n) \prec h(n) \Rightarrow f(n) \prec h(n)$$

$$f(n) \succeq g(n) \text{ and } g(n) \succeq h(n) \Rightarrow f(n) \succeq h(n)$$

$$f(n) \succ g(n) \text{ and } g(n) \succ h(n) \Rightarrow f(n) \succ h(n).$$

4. Transpose symmetry:

$$f(n) \succeq g(n) \Leftrightarrow g(n) \preceq f(n)$$

$$f(n) \succ g(n) \Leftrightarrow g(n) \prec f(n).$$

$$10. \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L \Rightarrow f(n) \approx g(n)$$

$$f(n) \approx g(n) \not\Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{q(n)} \text{ exists.}$$

$$5. f(n) \prec g(n) \Rightarrow f(n) \preceq g(n)$$

$$f(n) \preceq g(n) \not\Leftrightarrow f(n) \prec g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \succeq g(n)$$

$$f(n) \succeq g(n) \not\Leftrightarrow f(n) \succ g(n)$$

$$6. f(n) \approx g(n) \Rightarrow f(n) \not\prec g(n)$$

$$f(n) \approx g(n) \Rightarrow f(n) \not\succ g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\approx g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\succeq g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\preceq g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\approx g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\prec g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\preceq g(n)$$

$$7. f(n) \approx g(n) \Leftrightarrow f(n) \preceq g(n) \text{ and } f(n) \succeq g(n)$$

$$8. f(n) \prec g(n) \text{ and } f(n) \succ g(n) \text{ mümkün değil}$$

$$9. \text{ Let } f(n) = f_1(n) \pm f_2(n) \pm f_3(n) \pm \dots \pm f_k(n). \text{ Let}$$

$$f_1(n) \succ f_2(n)$$

$$f_1(n) \succ f_3(n)$$

...

$$f_1(n) \succ f_k(n)$$

Then $f(n) \approx f_1(n)$.

$$f(n) = \frac{1}{2}n^2 - 3n. \qquad f(n) \approx n^2.$$

$$0 \leq c_1.n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2.n^2$$

$$0 \leq c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

$$0 \leq c_1$$

$$c_1 \leq \frac{1}{2} - \frac{3}{n}$$

$$\frac{1}{2} - \frac{3}{n} \leq c_2$$

$$c_1 > 0 \quad n'_0 = 1 \quad c_2 = 1.$$

$$\frac{1}{2} - \frac{3}{7} = \frac{7}{14} - \frac{6}{14} = \frac{1}{14}.$$

$$c_1 = \frac{1}{14} \text{ and } n''_0 = 7.$$

$$n_0 = \max\{n'_0, n''_0\} = 7.$$

$$n_0 = 7, c_1 = \frac{1}{14}, c_2 = 1$$

Hangisi doğrudur?

$$2^{n+1} \approx 2^n$$

$$2^{2n} \approx 2^n$$

$$2^{n+1} = 2 \cdot 2^n$$

$$c \cdot 2^n \approx 2^n.$$

$$2^{n+1} \approx 2^n$$

Which of the following are true:

$$\frac{1}{n^2} \prec \frac{1}{n} \quad \text{Doğru}$$

$$2^{\frac{1}{n^2}} \prec 2^{\frac{1}{n}} \quad \text{eşit}$$

$$\lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n^2}}}{2^{\frac{1}{n}}} \right) = \lim_{n \rightarrow \infty} \left(2^{\frac{1}{n^2} - \frac{1}{n}} \right) = 1 \text{ because}$$

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n^2} - \frac{1}{n} \right) = \lim_{n \rightarrow \infty} \left(\frac{1-n}{n^2} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n} - 1}{n} \right) = 0$$

Hangisi doğrudur?

$$\text{Doğru} \quad \frac{1}{n} \prec 1 - \frac{1}{n}$$

$$\text{eşit} \quad 2^{\frac{1}{n}} \prec 2^{1-\frac{1}{n}}$$

$$\lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n}}{1 - \frac{1}{n}} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n}}{\frac{n-1}{n}} \right) = \lim_{n \rightarrow \infty} \frac{1}{n-1} = 0$$

$$\lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n}}}{2^{1-\frac{1}{n}}} \right) = \lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n}}}{2^1} \right) = \text{const}$$

$$2^{2n} = 2^n \cdot 2^n$$

$$2^{2n} \approx 2^n$$

not true.

$$2^n \cdot 2^n \leq c_2 \cdot 2^n \Leftrightarrow 2^n \leq c_2$$

Hangisi doğrudur ?

$$f(n) \approx g(n) \Rightarrow a^{f(n)} \approx a^{g(n)}$$

$$a > 1 \quad f(n) \preceq g(n) \Rightarrow a^{f(n)} \preceq a^{g(n)}$$

$$f(n) \prec g(n) \Rightarrow a^{f(n)} \prec a^{g(n)}$$

Üçü de yanlış

$$2n \approx n \text{ and } 2^{2n} \not\approx 2^n.$$

$$\frac{1}{n} \prec 1 - \frac{1}{n} \text{ but } 2^{\frac{1}{n}} \not\prec 2^{1-\frac{1}{n}}.$$

$$\frac{1}{n^2} \prec \frac{1}{n} \text{ but } 2^{\frac{1}{n^2}} \not\prec 2^{\frac{1}{n}}.$$

$$\begin{array}{ll}
 a^{f(n)} \approx a^{g(n)} \Rightarrow f(n) \approx g(n) & \text{Doğru} \\
 a > 1. \quad a^{f(n)} \preceq a^{g(n)} \Rightarrow f(n) \preceq g(n) & \text{Doğru} \\
 a^{f(n)} \prec a^{g(n)} \Rightarrow f(n) \prec g(n) & \text{Yanlış}
 \end{array}
 \longrightarrow
 \begin{array}{l}
 g(n) = \frac{1}{n}. \\
 2^{1-\frac{1}{n}} \approx 2^{\frac{1}{n}}, \\
 2^{1-\frac{1}{n}} \preceq 2^{\frac{1}{n}}, \text{ but } \frac{1}{n} \prec 1 - \frac{1}{n}, \\
 1 - \frac{1}{n} \not\prec \frac{1}{n}. \\
 2^{\frac{1}{n^2}} \preceq 2^{\frac{1}{n}} \\
 \frac{1}{n^2} \not\prec \frac{1}{n}.
 \end{array}$$

$$2^n \prec 2^{2n} \text{ but } n \not\prec 2n.$$

$$a^n \prec n! \prec n^n$$

$$\lim_{n \rightarrow \infty} \left(\frac{n \cdot (n-1) \cdot (n-2) \dots 2 \cdot 1}{\underbrace{a \cdot a \cdot a \dots a \cdot a}_{n \text{ times}}} \right) = \infty$$

$$\lim_{n \rightarrow \infty} \left(\frac{n \cdot (n-1) \cdot (n-2) \dots 2 \cdot 1}{\underbrace{n \cdot n \cdot n \dots n \cdot n}_{n \text{ times}}} \right) = 0$$

a , k and ϵ be any constants, $k > 0$, $a > 1$, and $\epsilon > 0$.

$$(\log_a n)^k \prec n^\epsilon \quad \text{Doğru}$$

a and ϵ be any constants, $a > 1$ and $\epsilon > 0$.

$$n^\epsilon \prec a^n \quad \text{Doğru}$$

Her iki tarafın a tabanına göre logaritmasını alalım

$$\epsilon \cdot \log_a n \prec n$$

$$\epsilon \cdot \log_a n \prec n$$

$$\lim_{n \rightarrow \infty} \frac{n^\epsilon}{(\log_a n)^k} =$$

$$b \leftarrow \frac{\epsilon}{k} \quad \text{olsun}$$

$$\lim_{n \rightarrow \infty} \frac{(n^b)^k}{(\log_a n)^k} =$$

$$\lim_{n \rightarrow \infty} \left(\frac{n^b}{\log_a n} \right)^k =$$

k positive

$$\lim_{n \rightarrow \infty} \frac{n^b}{\log_a n} = \quad \text{l'Hôpital's rule uygulansın}$$

$$\lim_{n \rightarrow \infty} \frac{bn^{b-1}}{\left(\frac{1}{\ln a}\right) \left(\frac{1}{n}\right)} = \lim_{n \rightarrow \infty} (\ln a) b n^b = \infty$$

Definition (log-star function)

$$\lg^{(i)} n = \begin{cases} n, & \text{if } i = 0 \\ \lg \left(\lg^{(i-1)} n \right), & \text{if } i > 0 \text{ and } \lg^{(i-1)} n > 0 \\ \text{undefined}, & \text{if } i > 0 \text{ and } \lg^{(i-1)} n < 0 \text{ or } \lg^{(i-1)} n \text{ is undefined} \end{cases}$$

$$\lg^* n = \min \left\{ i \geq 0 \mid \lg^{(i)} n \leq 1 \right\}$$

$$\lg^* 2 = 1, \text{ since } \lg^{(0)} 2 = 2 \text{ and } \lg^{(1)} 2 = \lg \left(\lg^{(0)} 2 \right) = \lg (2) = 1$$

$$\lg^* 3 = 2, \text{ since } \lg^{(0)} 3 = 3 \text{ and } \lg \left(\lg^{(0)} 3 \right) = \lg (\lg 3) = 0.6644 \dots$$

$$\lg^* 4 = 2$$

$$\lg^* 5 = 3$$

...

$$\lg^* 16 = 3$$

$$\lg^* 17 = 4$$

...

$$\lg^* 65536 = 4$$

$$\lg^* 65537 = 5$$

...

$$\lg^* 2^{65536} = 5$$

$$\lg^* \left(2^{65536} + 1 \right) = 6$$

...

Her gerçel sayı t, ikililerden oluşan bir kule ile temsil edilebilir:

$$t = 2^{2^{2^{\cdot^{\cdot^{\cdot^2^s}}}}}$$

Burada s, $1 < s \leq 2$ koşulunu sağlayan bir gerçel sayıdır. Kulenin yüksekliği, bu dizideki eleman sayısıdır.

<i>number</i>	<i>its tower of twos</i>	<i>the height of the tower</i>
2	2	1
3	$2^{1.5849625007\dots}$	2
4	2^2	2
5	$2^{2^{1.2153232957\dots}}$	3
16	2^{2^2}	3
17	$2^{2^{2^{1.0223362884\dots}}}$	4
65536	$2^{2^{2^2}}$	4
65537	$2^{2^{2^{2^{1.000000051642167\dots}}}}}$	5

Bunu göz önünde bulundurarak, lg n'nin n adet ikili kulenin yüksekliği olduğunu görmek çok kolaydır.

ÖRNEKLER

$$2^{2^{n+1}} \succ 2^{2^n} \quad 2^{2^{n+1}} = 2^{2 \cdot 2^n} = 2^{2^n} \times 2^{2^n}.$$

$$(n+1)! \succ n!$$

$$(n+1)! = (n+1) \times n!$$

$$2^{2^n} \succ (n+1)! \quad \text{Her iki tarafın logaritması alınırsa}$$

$$2^n \lg((n+1)!)$$

$$\lg((n+1)!) \approx (n+1) \lg(n+1)$$

$$(n+1) \lg(n+1) \approx n \lg n.$$

$$2^n \succ n \lg n,$$

$$2^{2^n} \succ (n+1)!$$

$$e^n \succ n \cdot 2^n$$

$$\lim_{n \rightarrow \infty} \frac{n \cdot 2^n}{e^n} = \lim_{n \rightarrow \infty} \frac{n}{\frac{e^n}{2^n}} = \lim_{n \rightarrow \infty} \frac{n}{\left(\frac{e}{2}\right)^n} = 0$$

$$n \cdot 2^n \succ 2^n$$

$$2^n \succ \left(\frac{3}{2}\right)^n \quad \lim_{n \rightarrow \infty} \frac{\left(\frac{3}{2}\right)^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{3}{4}\right)^n = 0$$

$$\left(\frac{3}{2}\right)^n \succ n^{\lg(\lg n)} \quad \text{Heriki tarafın logaritması alınır}$$

$$(\lg n)^{\lg n} \approx n^{\lg(\lg n)}$$

Heriki tarafın logaritması alınır

$$n \cdot \lg\left(\frac{3}{2}\right) \succ \lg n \cdot \lg(\lg n)$$

$$(\lg n) \cdot (\lg(\lg n)) \approx (\lg(\lg n)) \cdot (\lg n)$$

$$n \succ \lg^2 n$$

$$n \succ \lg n \cdot \lg(\lg n)$$

$$\lg^2 n \succ \lg n \cdot \lg(\lg n)$$

$$(\lg n)^{\lg n} \succ (\lg n)!$$

$$m = \lg n \quad \text{olsun} \quad m^m \succ m!$$

$$(\lg n)! \succ n^3$$

Heriki tarafın logaritması alınır

$$m = \lg n \quad \text{olsun} \quad \lg((\lg n)!) \quad 3 \cdot \lg n$$

$$\lg(m!)$$

$$m \lg m, \succ 3 \cdot \lg n$$

$$\lim_{n \rightarrow \infty} \frac{3 \cdot \lg n}{(\lg n) \cdot (\lg(\lg n))} = \lim_{n \rightarrow \infty} \frac{3}{\lg(\lg n)} = 0$$

$$\lg((\lg n)!) \approx (\lg n) \cdot (\lg(\lg n)).$$

$$n^3 \succ n^2.$$

$$n^2 \approx 4^{\lg n}$$

$$n^2 \succ n \lg n.$$

$$\lg n! \approx n \lg n$$

$$n \lg n \succ n.$$

$$n \approx 2^{\lg n}$$

$$n \succ (\sqrt{2})^{\lg n}$$

$$(\sqrt{2})^{\lg n} = 2^{\frac{1}{2} \lg n} = 2^{\lg \sqrt{n}} = \sqrt{n}$$

$$n \succ \sqrt{n}.$$

$$(\sqrt{2})^{\lg n} \succ 2^{\sqrt{2} \lg n}$$

$$\lg n \succ \sqrt{\lg n},$$

$$\frac{1}{2} \cdot \lg n \succ \sqrt{2} \cdot \sqrt{\lg n} = \sqrt{2 \lg n}.$$

$$2^{\frac{1}{2} \cdot \lg n} \succ 2^{\sqrt{2} \lg n}$$

$$(\sqrt{2})^{\lg n} \succ 2^{\sqrt{2} \lg n}.$$

$$2^{\sqrt{2} \lg n} \succ \lg^2 n.$$

Heriki tarafta logaritma alalım

$$\sqrt{2 \lg n} \quad \lg (\lg^2 n) = 2 \cdot \lg (\lg n)$$

$m = \lg n$ olsun

$$\sqrt{2m} = \sqrt{2} \sqrt{m} = \sqrt{2} \cdot m^{\frac{1}{2}} \quad 2 \lg m.$$

$$m^{\frac{1}{2}} \succ \lg m$$

$$\sqrt{2} \cdot m^{\frac{1}{2}} \succ 2 \lg m,$$

$$\sqrt{2m} \succ 2 \lg m$$

$$\sqrt{2 \lg n} \succ \lg (\lg^2 n).$$

$$\lg^2 n \succ \ln n.$$

$$\ln n \succ \sqrt{\lg n}.$$

$$\sqrt{\lg n} \succ \ln \ln n$$

$$\ln \ln n \succ 2^{\lg^* n}$$

$$\ln n = \frac{\lg n}{\lg e}.$$

$$\sqrt{\frac{\ln n}{\ln 2}} \succ \ln \ln n$$

$\ln \ln n \approx \lg \lg n$ olduğundan
yeniden aşağıdaki gibi yazılabilir

$$m = \ln n \text{ olsun}$$

$$\lg \lg n \succ 2^{\lg^* n}$$

Heriki tarafın logaritmasını alalım

$$\frac{1}{\sqrt{\ln 2}} \cdot \sqrt{m} \succ \ln m$$

$$\lg \lg \lg n \succ \lg^* n.$$

$$2^{\lg^* n} \succ \lg^* n$$

$$\lg^* n \approx \lg^* (\lg n)$$

$f(n)$ artan bir fonksiyon ise

Hatırlatma: n sayısı 2'ler kulesi olsun.

n ile $\lg n$ 'inkule yüksekikleri arasındaki fark 1'dir.

$$2^{f(n)} \succ f(n)$$

$$\lg^* (\lg n) = (\lg^* n) - 1$$

$$1 \approx n^{\frac{1}{\lg n}}$$

$$\lg^* n \succ \lg (\lg^* n)$$

$$n^{\frac{1}{\lg n}} = 2$$

$$\lg \left(n^{\frac{1}{\lg n}} \right) = \frac{1}{\lg n} \cdot \lg n = 1$$

$$\lg (\lg^* n) \succ 1$$

k bir sabit ve $0 < k < 1$ ise

$$1 + k + k^2 + k^3 + \dots + k^n = \Theta(1)$$

$$1 + k + k^2 + k^3 + \dots + k^n < \sum_{t=0}^{\infty} k^t = \frac{1}{1-k} = \Theta(1)$$

Genelleme:

$$1 + k + k^2 + k^3 + \dots + k^n = \begin{cases} \Theta(1), & \text{if } 0 < k < 1 \\ \Theta(n), & \text{if } k = 1 \\ \Theta(k^n), & \text{if } k > 1 \end{cases}$$

Algoritma Analizi ve Tasarımı

Dinamik Programlama

Dinamik Programlama

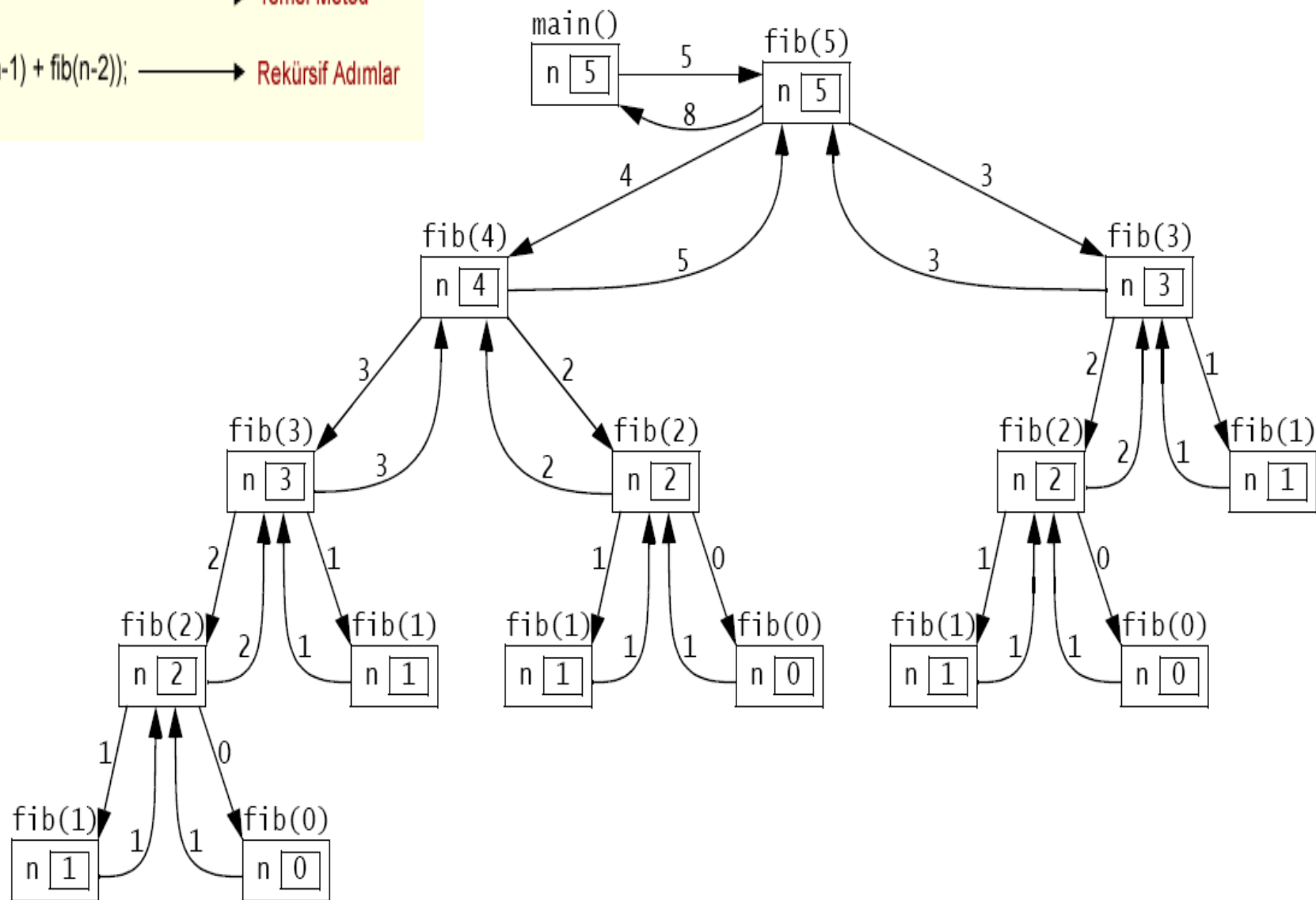
- Algoritma tasarım tekniği-Divide and conquer
- (Böl ve Yönet)
 - Problem bağımsız alprogramlar halinde bölünür
 - Altprogramlar rekürsif olarak çözülür
 - Orjinal problemi çözmek için çözümler birleştirilir


```

int fib(int n) {
    if (n <= 1)
        return n;
    else
        return (fib(n-1) + fib(n-2));
}

```

→ Temel Metod
 → Rekürsif Adımlar



```
1    public static int fib(int n) {  
2        if (n < 2) {  
3            return n;  
4        }  
5        int[] f = new int[n];  
6        f[0] = 0;  
7        f[1] = 1;  
8        for (int i=2; i<n; i++) {  
9            f[i] = f[i-1] + f[i-2];  
10       }  
11       return f[n-1] + f[n-2];  
12   }
```

Dinamik Programlama

- Alt problemler bağımsızken uygulanabilir.
 - Altproblemler daha küçük altpromlarla da paylaşılır.

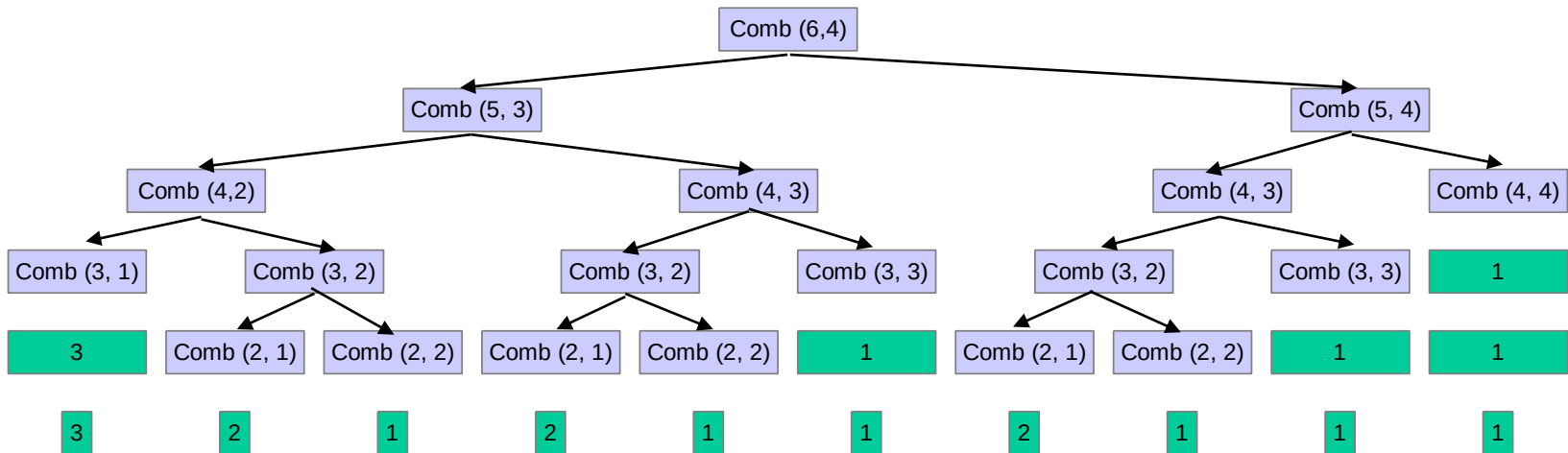
Combinations:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\binom{n}{1} = 1 \qquad \binom{n}{n} = 1$$

- Böl ve Yönet yaklaşımı ortak alt problemleri tekrar tekrar çözecektir
- Dinamik programlama her alt problemi sadece bir kez çözer ve cevabı bir tabloda saklar

Kombinasyon Problemi



$$\begin{pmatrix} n \\ k \end{pmatrix} = \begin{pmatrix} n-1 \\ k \end{pmatrix} + \begin{pmatrix} n-1 \\ k-1 \end{pmatrix}$$

Recurrence: $C(n,k) = C(n-1,k) + C(n-1,k-1)$ for $n > k > 0$
 $C(n,0) = 1, C(n,n) = 1$ for $n \geq 0$

		k 							
		0	1	2	3	4	5	6	7
n 	0	1							
	1	1	1						
	2	1	2	1					
	3	1	3	3	1				
	4	1	4	6	4	1			
	5	1	5	10	10	5	1		
	6	1	6	15	20	15	6	1	
	7	1	7	21	35	35	21	7	1
	8	1	8	28	56	70	56	28	8
	9	1	9	36	84	126	126	84	36
	10	1	10	45	120	210	252	210	120

```
for  $i \leftarrow 0$  to  $n$  do  
    for  $j \leftarrow 0$  to  $\min(i, k)$  do  
        if  $j = 0$  or  $j = i$   
             $C[i, j] \leftarrow 1$   
        else  $C[i, j] \leftarrow C[i - 1, j - 1] + C[i - 1, j]$   
return  $C[n, k]$ 
```

Time efficiency: $\Theta(nk)$

Space efficiency: $\Theta(nk)$

```

// A Dynamic Programming based solution to compute nCr % p
#include <bits/stdc++.h>
using namespace std;
// Returns nCr % p
int nCrModp(int n, int r, int p)
{
    // Optimization for the cases when r is large
    if (r > n - r)
        r = n - r;

    // The array C is going to store last row of pascal triangle at the end. And last entry of last row is nCr
    int C[r + 1];
    memset(C, 0, sizeof(C));

    C[0] = 1; // Top row of Pascal Triangle

    // One by constructs remaining rows of Pascal Triangle from top to bottom
    for (int i = 1; i <= n; i++) {
        // Fill entries of current row using previous row values
        for (int j = min(i, r); j > 0; j--)
            // nCj = (n-1)Cj + (n-1)C(j-1);
            C[j] = (C[j] + C[j - 1]) % p;
    }
    return C[r];
}

// Driver program
int main()
{
    int n = 10, r = 2, p = 13;
    cout << "Value of nCr % p is " << nCrModp(n, r, p);
    return 0;
}

```

Dinamik Programlama

- **Optimizasyon problemleri**

- En iyi çözümü elde etmek için bir dizi seçenek yapılmalıdır
- En uygun değere sahip bir çözüm bulunur (minimum veya maksimum)
- Optimum değere götüren birçok çözüm olabilir
- Hedef: En iyi çözümü bulmak

Dinamik Programlama Algoritması

1. Optimal bir çözümün yapısını karakterize edin
2. Optimal bir çözümün değerini tekrar tekrar tanımlayın
3. En uygun çözümün değerini aşağıdan yukarıya doğru hesaplayın
4. Hesaplanan bilgilerden optimal bir çözüm oluşturun

Matris-Zinciri Çarpımı

Problem: verilen bir dizi $\langle A_1, A_2, \dots, A_n \rangle$, matrisi için aşağıdaki çarpımın hesaplanması.

$$A_1 \cdot A_2 \cdots A_n$$

- Matris uyumluluğu:

$$C = A \cdot B$$

$$\text{col}_A = \text{row}_B$$

$$\text{row}_C = \text{row}_A$$

$$\text{col}_C = \text{col}_B$$

$$C = A_1 \cdot A_2 \cdots A_i \cdot A_{i+1} \cdots A_n$$

$$\text{col}_i = \text{row}_{i+1}$$

$$\text{row}_C = \text{row}_{A_1}$$

$$\text{col}_C = \text{col}_{A_n}$$

MATRIX-MULTIPLY(A, B)

if $\text{columns}[A] \neq \text{rows}[B]$

then error "incompatible dimensions"

else for $i \leftarrow 1$ to $\text{rows}[A]$

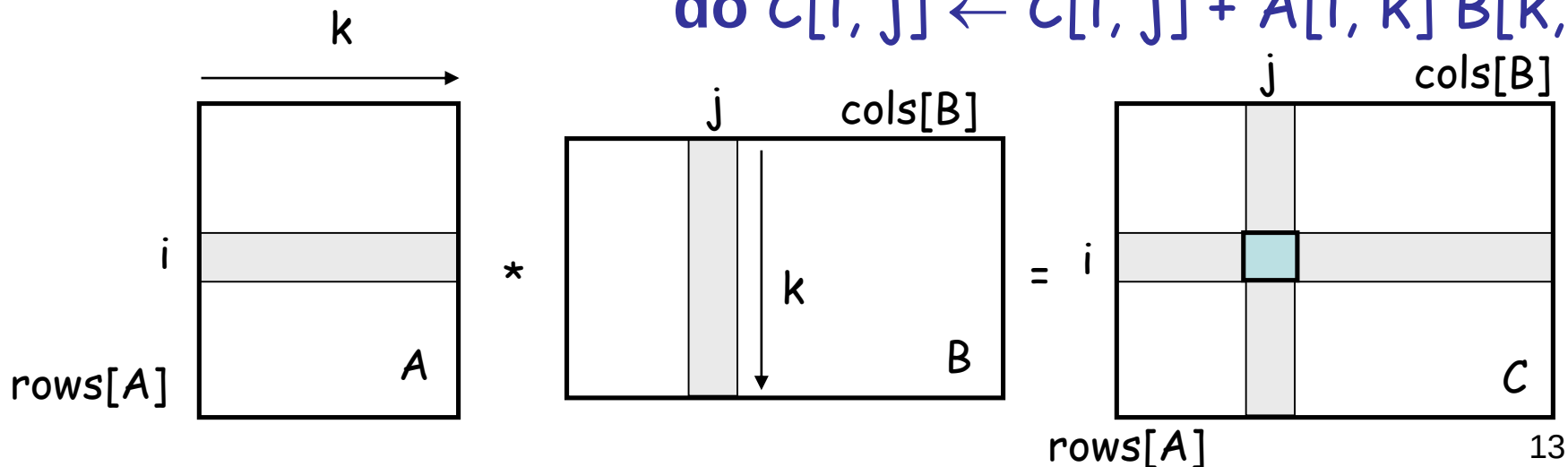
do for $j \leftarrow 1$ to $\text{columns}[B]$

do $C[i, j] = 0$

for $k \leftarrow 1$ to $\text{columns}[A]$

do $C[i, j] \leftarrow C[i, j] + A[i, k] B[k, j]$

$\text{rows}[A] \cdot \text{cols}[A] \cdot \text{cols}[B]$
multiplications



Matris-ZinciriÇarpımı

- Matrisleri hangi sırayla çarpmalıyız?

$$A_1 \cdot A_2 \cdots A_n$$

- Matrislerin çarpılacağı sırayı almak için ürünü parantez içine alalım:

$$\begin{aligned}\text{Şöyleki: } A_1 \cdot A_2 \cdot A_3 &= ((A_1 \cdot A_2) \cdot A_3) \\ &= (A_1 \cdot (A_2 \cdot A_3))\end{aligned}$$

- Bu siparişlerden hangisini seçmeliyiz?
- Matrisleri çarpma düzenimiz, çarpımı değerlendirme maliyeti üzerinde önemlidir.

Örnek

$$A_1 \cdot A_2 \cdot A_3$$

- A_1 : 10×100
- A_2 : 100×5
- A_3 : 5×50

1. $((A_1 \cdot A_2) \cdot A_3)$: $A_1 \cdot A_2 = 10 \times 100 \times 5 = 5,000$ (10×5)
 $((A_1 \cdot A_2) \cdot A_3) = 10 \times 5 \times 50 = 2,500$

Total: 7,500 skaler çarpım

2. $(A_1 \cdot (A_2 \cdot A_3))$: $A_2 \cdot A_3 = 100 \times 5 \times 50 = 25,000$ (100×50)
 $(A_1 \cdot (A_2 \cdot A_3)) = 10 \times 100 \times 50 = 50,000$

Total: 75,000 skaler çarpım

Matris-ZinciriÇarpımı Problemi

- Verilen bir matris zincirinde $\langle A_1, A_2, \dots, A_n \rangle$, burada A_i , $p_{i-1} \times p_i$ boyutlarındadır. Skaler çarpım sayısını minimuma indirecek şekilde $A_1 \cdot A_2 \cdots A_n$ çarpımını tam parantezlersek.

$$\begin{array}{ccccccc} A_1 & \cdot & A_2 & \cdots & A_i & \cdot & A_{i+1} & \cdots & A_n \\ p_0 \times p_1 & & p_1 \times p_2 & & p_{i-1} \times p_i & & p_i \times p_{i+1} & & p_{n-1} \times p_n \end{array}$$

Olası parentezleme sayısı nedir?

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k). \qquad f(n) = \frac{1}{n} \binom{2n-2}{n-1}.$$

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796,

4 adet matrisin çarpımı için 5; 10 adet matris için 4862 farklı çarpım düzeni vardır.

$$f(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi} n^{1.5}}.$$

$$f(n) = \Omega\left(\frac{4^n}{n^{1.5}}\right).$$

$$\Omega(4^n/n^{3/2})$$

1. Optimal parantezleme yapısı

- Gösterim:

$$A_{i\dots j} = A_i A_{i+1} \cdots A_j, i \leq j$$

- Diyelim ki $A_{i\dots j}$ için bir optimal parentezleme ifadeyi, A_k ve A_{k+1} , arasında bölsün.
- Burada $i \leq k < j$

$$\begin{aligned} A_{i\dots j} &= A_i A_{i+1} \cdots A_j \\ &= A_i A_{i+1} \cdots A_k A_{k+1} \cdots A_j \\ &= A_{i\dots k} A_{k+1\dots j} \end{aligned}$$

Optimal altproblem

$$A_{i\dots j} = A_{i\dots k} A_{k+1\dots j}$$

- $A_{i\dots k}$ kesinlikle optimal parentezleme olmalı
- $A_{i\dots k}$, için daha az maliyetli bir parentezleme olsaydı, Bunu $A_{i\dots j}$ 'ninin parantez içindeki yerine koyabilir ve optimumdan daha düşük maliyetli bir parantez oluşturabilirdik. Bu da bir çelişki olacaktır.
- **Matris zinciri çarpımının bir örneğine en uygun çözüm, içinde alt sorunlara en uygun çözümleri içerir**

2. Özyineli bir çözüm

- Altproblem:

Minimum parantezleme maliyeti belirlenir.

$$A_{i\dots j} = A_i A_{i+1} \cdots A_j \quad \text{for } 1 \leq i \leq j \leq n$$

- $m[i, j] = A_{i\dots j}$ 'yi hesaplamak için gereken minimum çarpma sayısı
- Ana problem ($A_{1..n}$): $m[1, n]$
 - $i = j$: $A_{i\dots i} = A_i \Rightarrow m[i, i] = 0$, for $i = 1, 2, \dots, n$

2. Özyineli bir çözüm

- Parantezleme altproblemini ele aldığımızda

$$A_{i...j} = A_i A_{i+1} \cdots A_j \quad \text{for } 1 \leq i \leq j \leq n$$

$$= \underbrace{A_{i...k}}_{m[i,k]} \underbrace{A_{k+1...j}}_{m[k+1,j]} \quad \text{for } i \leq k < j$$

$p_{i-1}p_kp_j$

- Eğer optimal parantezleme çarpımı aşağıdaki gibi bölse $A_i A_{i+1} \cdots A_j$ k , ($i \leq k < j$) arasında

$$m[i, j] = \underbrace{m[i, k]} + \underbrace{m[k+1, j]} + \underbrace{p_{i-1}p_kp_j}$$

$A_{i...k}$ hesabı için
minimum çarpma
sayısı

$A_{k+1...j}$ hesabı için
minimum çarpma
sayısı

$A_{i...k}A_{k...j}$ hesabı için
minimum çarpma
sayısı

2. Özyineli bir çözüm

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$$

- k değerini bilmiyoruz.
 - k için $j - i$ muhtemel değer var $k: k = i, i+1, \dots, j-1$
- $A_i A_{i+1} \dots A_j$ 'yi Parantezleme maliyetini minimuma indirme problemi şu hale gelecektir.

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

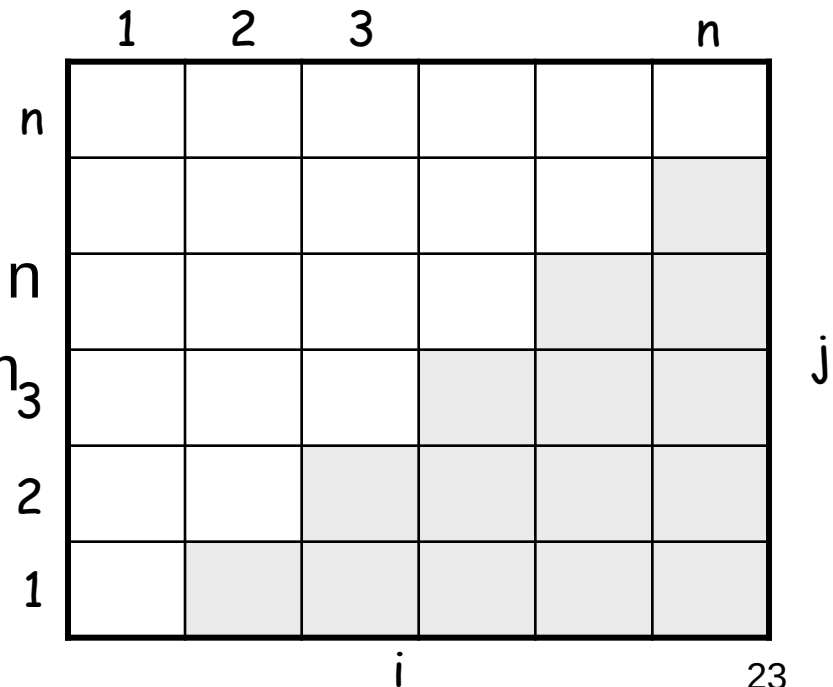
3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Optimal çözümü rekürsif olarak hesaplamak üstel zaman maliyeti alır!
- Alt problem sayısı?

$$\Rightarrow \Theta(n^2)$$

- $A_{i...j}$ 'yi parentezle $1 \leq i \leq j \leq n$
- Her i ve j seçimi 1 problem



3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Bu $m[1..n, 1..n]$ tablosunu nasıl dolduracağız?
 - $m[i, j]$ hesabında hangi tablo girişleri kullanılacaktır?

$$A_{i...j} = A_{i...k} A_{k+1...j}$$

- Alt problemlerin boyutu orijinal boyuttan küçük
- **Idea:** m 'yi artan uzunluktaki problemleri çözecek şekilde doldurun

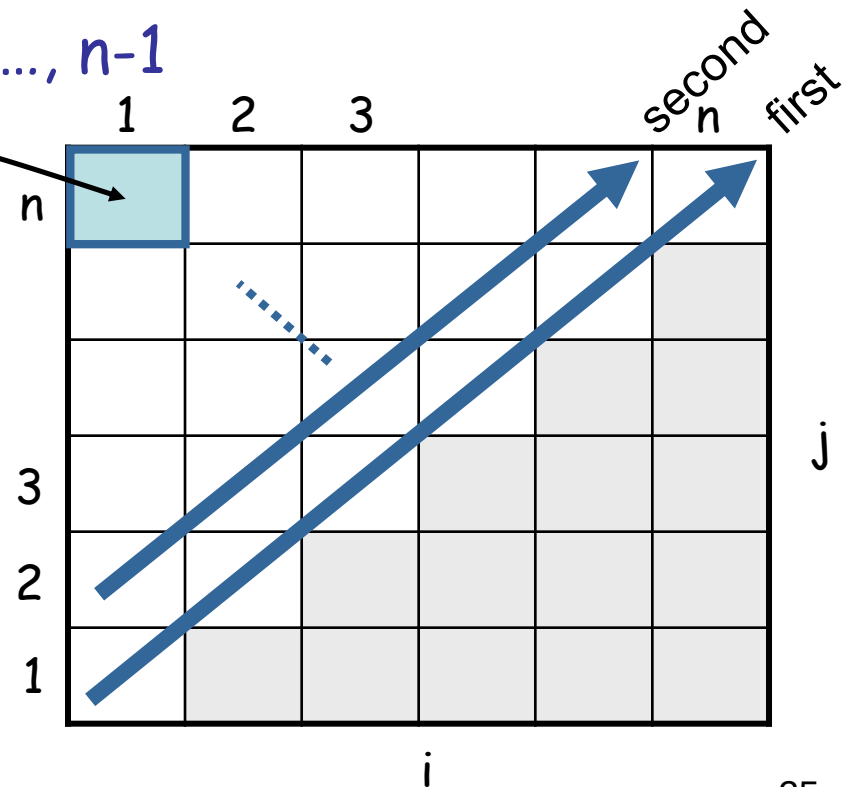
3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Length = 1: $i = j, i = 1, 2, \dots, n$
- Length = 2: $j = i + 1, i = 1, 2, \dots, n-1$

$m[1, n]$:problemin
optimal çözümü

Satırlar aşağıdan yukarıya ve
soldan sağa hesaplanır.



Örnek: $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1p_2p_5 & k = 2 \\ m[2, 3] + m[4, 5] + p_1p_3p_5 & k = 3 \\ m[2, 4] + m[5, 5] + p_1p_4p_5 & k = 4 \end{cases}$$

	1	2	3	4	5	6
6						
5						
4						
3						
2						
1						

i

$m[i, j]$ değerleri sadece
önceden hesaplanmış
değerlere bağlıdır.

Örnek $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

$A_1 \cdot A_2 \cdot A_3$ çarpımını hesapla:

- A_1 : 10×100 ($p_0 \times p_1$)
- A_2 : 100×5 ($p_1 \times p_2$)
- A_3 : 5×50 ($p_2 \times p_3$)

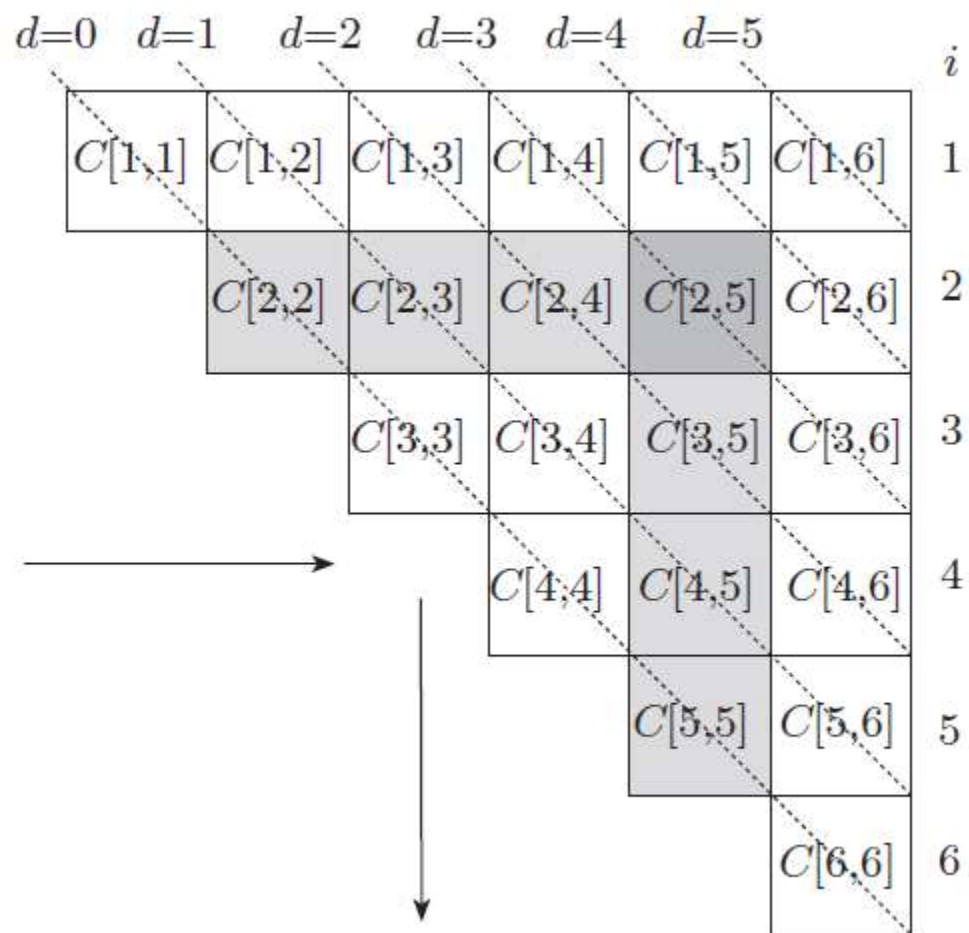
$m[i, i] = 0$ for $i = 1, 2, 3$

$$\begin{aligned} m[1, 2] &= m[1, 1] + m[2, 2] + p_0p_1p_2 && (A_1A_2) \\ &= 0 + 0 + 10 * 100 * 5 = 5,000 \end{aligned}$$

$$\begin{aligned} m[2, 3] &= m[2, 2] + m[3, 3] + p_1p_2p_3 && (A_2A_3) \\ &= 0 + 0 + 100 * 5 * 50 = 25,000 \end{aligned}$$

$$m[1, 3] = \min \begin{cases} m[1, 1] + m[2, 3] + p_0p_1p_3 = 75,000 & (A_1(A_2A_3)) \\ m[1, 2] + m[3, 3] + p_0p_2p_3 = 7,500 & ((A_1A_2)A_3) \end{cases}$$

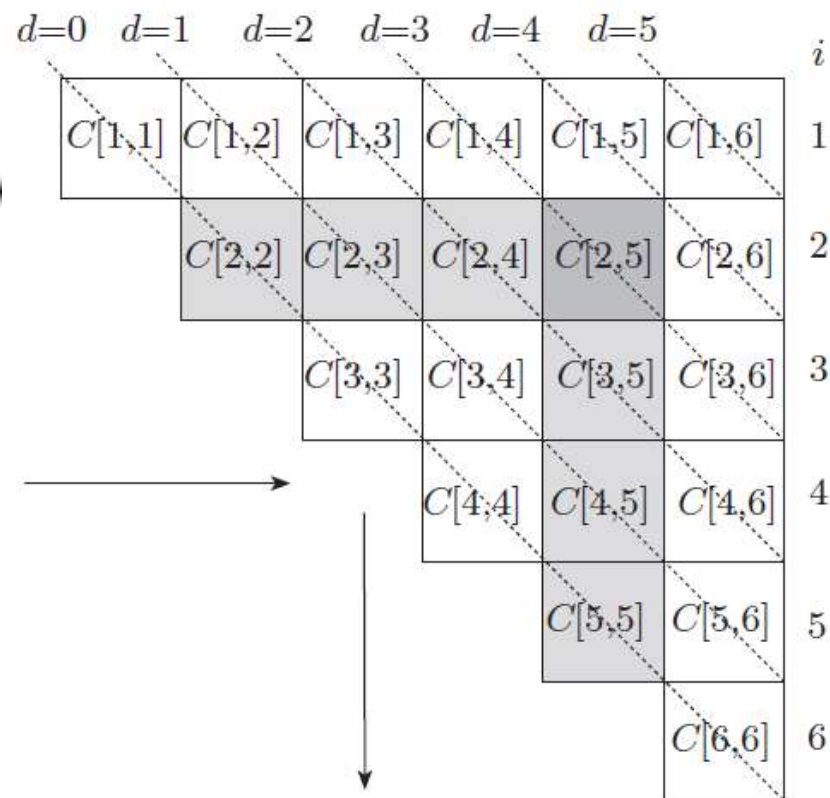
	1	2	3
3	² 7500	² 25000	0
2	¹ 5000	0	
1	0		



```

1. for  $i \leftarrow 1$  to  $n$ 
2.    $C[i, i] \leftarrow 0$ 
3. end for
4. for  $d \leftarrow 1$  to  $n - 1$ 
5.   for  $i \leftarrow 1$  to  $n - d$ 
6.      $j \leftarrow i + d$ 
7.     comment: The next three lines compute  $C[i, j]$ 
8.      $C[i, j] \leftarrow \infty$ 
9.     for  $k \leftarrow i + 1$  to  $j$ 
10.       $C[i, j] \leftarrow \min\{C[i, j], C[i, k - 1] + C[k, j] + r[i]r[k]r[j + 1]\}$ 
11.    end for
12.  end for
13. end for
14. return  $C[1, n]$ 

```



Matrix-Chain-Order(p)

```
MATRIX-CHAIN-ORDER(p)
1  n ← length[p] − 1
2  for i ← 1 to n
3      do m[i, i] ← 0
4  for l ← 2 to n           ▷ l is the chain length.
5      do for i ← 1 to n − l + 1
6          do j ← i + l − 1
7              m[i, j] ← ∞
8              for k ← i to j − 1
9                  do q ← m[i, k] + m[k + 1, j] + pi−1pkpj
10                 if q < m[i, j]
11                     then m[i, j] ← q
12                     s[i, j] ← k
13  return m and s
```

$O(N^3)$

4. Optimal Çözümü Yapılandır

- Benzer bir matriste k 'nin optimal değerlerini saklayalım
- $s[i, j] = k$;
- k , $A_{i..j}$ zincirini A_k ve A_{k+1} biçiminde çarpanlara ayırır.

	1	2	3		n
n					
			k		
3					
2					
1					

4. Optimal Çözümü Yapılandır

- $s[1, n]$, $A_{1..n}$ çarpımının tamamı ile ilişkilidir.
- Sonuç matris çarpımı $k=s[1, n]$ 'de bölünecektir.

$$A_{1..n} = A_{1..s[1, n]} \cdot A_{s[1, n]+1..n}$$

- Her bir alt çarpım için, optimum parantezlemeye karşılık gelen k değerini bulunur.

	1	2	3		n	
n						
3						
2						
1						

4. Optimal Çözümü Yapılandır

- $s[i, j] = A_i A_{i+1} \dots A_j$ zincirini A_k and A_{k+1} arasında çarpanlara ayıran k değeri

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

i

j

- $s[1, n] = 3 \Rightarrow A_{1..6} = A_{1..3} A_{4..6}$
- $s[1, 3] = 1 \Rightarrow A_{1..3} = A_{1..1} A_{2..3}$
- $s[4, 6] = 5 \Rightarrow A_{4..6} = A_{4..5} A_{6..6}$

4. Optimal Çözümü Yapılandır

PRINT-OPT-PARENS(s, i, j)

if $i = j$

then print " A_i "

else print "("

PRINT-OPT-PARENS($s, i, s[i, j]$)

PRINT-OPT-PARENS($s, s[i, j] + 1, j$)

print ")"

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					
	i					
						j

Örnek: $A_1 \cdot \cdot \cdot A_6$

$((A_1 (A_2 A_3)) ((A_4 A_5) A_6))$

PRINT-OPT-PARENS(s, i, j)

if $i = j$

then print " A_i "

else print "("

PRINT-OPT-PARENS($s, i, s[i, j]$)

PRINT-OPT-PARENS($s, s[i, j] + 1, j$)

print ")"

P-O-P($s, 1, 6$) $s[1, 6] = 3$

$i = 1, j = 6$ "(" P-O-P ($s, 1, 3$) $s[1, 3] = 1$

$i = 1, j = 3$ "(" P-O-P($s, 1, 1$) $\Rightarrow "A_1"$

P-O-P($s, 2, 3$) $s[2, 3] = 2$

$i = 2, j = 3$ "(" P-O-P ($s, 2, 2$) $\Rightarrow "A_2"$

P-O-P ($s, 3, 3$) $\Rightarrow "A_3"$

)"

)"

...

$s[1..6, 1..6]$

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

i

j

Memoization

- Tipik dinamik programlama yaklaşımının verimliliği ile yukarıdan aşağıya yaklaşım
- Her bir alt problemin çözümü için bir tabloya giriş sağlama
- **memoize** verimsiz özyinelemeli algoritmayı hatırlamak
- Bir alt problemle ilk karşılaşıldığında çözümü hesaplanır ve bu tabloda saklanır
- Alt probleme sürekli “çağrılar” ve sadece o anki değere bakın

Alg.: MEMOIZED-MATRIX-CHAIN(p)

1. $n \leftarrow \text{length}[p] - 1$

2. **for** $i \leftarrow 1$ **to** n

3. **do for** $j \leftarrow i$ **to** n

4. **do** $m[i, j] \leftarrow \infty$

} m tablosunu, $m[i, j]$ değerlerinin hesaplanıp hesaplanmadığını gösteren büyük değerlerle başlatın

5. **return** LOOKUP-CHAIN(p, 1, n) ← Top-down yaklaşım

Alg.: LOOKUP-CHAIN(p, i, j)

Running time is $O(n^3)$

1. **if** $m[i, j] < \infty$
2. **then return** $m[i, j]$
3. **if** $i = j$
4. **then** $m[i, j] \leftarrow 0$
5. **else for** $k \leftarrow i$ **to** $j - 1$
6. **do** $q \leftarrow$ LOOKUP-CHAIN(p, i, k) +
 LOOKUP-CHAIN($p, k+1, j$) + $p_{i-1}p_kp_j$
7. **if** $q < m[i, j]$
8. **then** $m[i, j] \leftarrow q$
9. **return** $m[i, j]$

Dinamik Programlama ↗ Hafıza

- Dinamik programlamanın kaydedilmiş algoritmalara göre avantajları
- Özyineleme için ek yük yok, tabloyu korumak için daha az ek yük
- Zaman veya alan gereksinimlerini azaltmak için tablo erişimine ilişkin normal desen kullanılabilir
- Kaydedilmiş algoritmaların dinamik programlamaya göre avantajları
- Bazı alt problemlerin çözülmesine gerek yoktur

Dinamik Programlamanın Bileşenleri

- Optimal AltProblem
 - Bir soruna optimal çözüm, alt sorunlara optimal çözümü içerir
 - Ana probleme optimal çözüm, optimal alt problem çözümlerden **aşağıdan yukarıya** doğru bir şekilde oluşturulur.
- Çakışan Alt Problemler(overlapping subproblems)
 - Eğer bir rekürsif algoritma aynı alt problemi tekrar tekrar çözme durumu

Optimal altyapının parametreleri

- Ana problem için optimal çözümde kaç alt problem kullanılmaktadır?
 - Matris çarpımı: İki altproblem ($A_{i..k}$, $A_{k+1..j}$)
- Optimum bir çözümde hangi alt sorunların kullanılacağını belirlemek için kaç seçeneğimiz var?
 - Matris çarpımı: $j - i$ seçenek k için (çarpımı ayırmak)

Optimal altyapının parametreleri

- Sezgisel olarak, bir dinamik programlama algoritmasının çalışma süresi iki faktöre bağlı:
 - Toplam altproblem sayısı
 - Her alt problem için kaç seçeneğe bakıyoruz?
- Matrix multiplication:
 - $\Theta(n^2)$ altproblem ($1 \leq i \leq j \leq n$)
 - En fazla $n-1$ seçim var

$\Theta(n^3)$ tamamı